# Programmation FPGA avec Eclat : comparaison de style et analyses des performances

Dahmane Ilan
APR/LIP6 et IRILL

1 Juin 2026 - 31 Juillet 2026

**Tuteur universitaire :**
Eugene Asarin
Giulio Manzonetto

**Tuteur de stage :**
Loïc Sylvestre
Emmanuel Chailloux

Université Paris Cité

SORBONNE UNIVERSITÉ

# Sommaire

**1 Introduction** **3**

**2 FPGA et programmation** **5**

**3 Expérimentations** **8**
3.1 Processeur LC-3 . . . . . 8
3.2 Implémentation du LC3 en Eclat . . . . . 9
3.3 Résultats des benchmarks . . . . . 10

**4 Conclusion** **12**

# 1 Introduction

Dans l'industrie informatique, les processeurs généralistes (CPU) constituent depuis plusieurs décennies la solution dominante pour l'exécution des programmes. Leur architecture est conçue pour offrir une grande flexibilité et pouvoir traiter une grande variété de tâches. Cependant, cette généralité limite leur efficacité lorsqu'il s'agit d'exploiter le parallélisme massif présent dans certaines applications.

Pour répondre à certains besoins spécifiques, les GPU (Graphics Processing Units) ont été développés initialement pour accélérer le rendu graphique. Leur architecture massivement parallèle les rend particulièrement efficaces pour les applications reposant sur de nombreuses opérations similaires appliquées sur de grands ensembles de données, comme le traitement d'image ou l'intelligence artificielle.

Cependant, toutes les applications ne se prêtent pas naturellement à ce modèle de calcul. Certaines nécessitent des architectures spécialisées capables d'exploiter un parallélisme différent, avec des contraintes fortes en termes de latence, de consommation énergétique ou de traitement temps réel. C'est dans ce contexte que les FPGA (Field Programmable Gate Array) constituent une solution intéressante grâce à leur capacité de reconfiguration [1] et leur aptitude à implémenter des architectures matérielles spécialisées.

Cependant, leur programmation reste un domaine complexe nécessitant traditionnellement une description matérielle bas niveau. Le langage Eclat [2] s'inscrit dans cette problématique en proposant une approche permettant de décrire des calculs matériels avec un niveau d'abstraction supérieur tout en conservant une traduction efficace vers des architectures FPGA.

Si les langages de plus haut niveau comme Eclat permettent de simplifier la programmation des FPGA, ils soulèvent néanmoins une question importante. Les concepteurs matériels raisonnent traditionnellement en termes de registres, de portes logiques, de mémoires et de cycles d'horloge, tandis que les programmeurs sont habitués à manipuler des abstractions de plus haut niveau, des structures de données complexes et différents paradigmes de programmation. Il devient alors nécessaire de déterminer quels styles de programmation permettent de concilier expressivité, sûreté et efficacité du matériel généré.

Dans quelle mesure les styles de programmation fonctionnel et impératif proposés par Eclat permettent-ils de décrire efficacement des algorithmes réalistes tout en générant des architectures matérielles performantes ?

Pour répondre à cette problématique, ce stage s'appuie notamment sur l'implémentation d'un processeur pédagogique LC-3 [1], étudié à l'Université Paris Cité dans le cadre du cours de Circuits et Architecture, enseigné par le professeur Adrien Guatto. Cette application sert de support à la comparaison des styles fonctionnel et impératif dans Eclat. Le processeur LC-3 constitue un excellent candidat pour cette évaluation grâce à son compromis entre simplicité

1. Le LC-3 est un processeur pédagogique conçu par Yale N.Patt et Sanjay J.Patel. Introduit dans leur ouvrage Introduction to Computing Systems afin d'enseigner les principes fondamentaux de l'architecture des ordinateurs.

et représentativité. Son architecture volontairement réduite permet une compréhension complète de son fonctionnement, tout en mettant en œuvre de nombreux mécanismes caractéristiques des processeurs modernes. Il sollicite ainsi plusieurs fonctionnalités importantes d'Eclat, telles que la gestion de l'état, des registres, de la mémoire et des structures de données. Cette complexité maîtrisée en fait un support pertinent pour tester un large panel des fonctionnalités du langage et comparer différentes approches d'implémentation. De plus, l'utilisation d'un même processeur comme base d'expérimentation permet de conserver un comportement fonctionnel identique entre les différentes versions, tout en observant l'impact du style de programmation choisi sur l'architecture matérielle générée, les performances obtenues et les ressources utilisées. Les deux styles sont évalués selon différents critères tels que la facilité d'écriture, les performances obtenues après synthèse et l'occupation des ressources matérielles sur FPGA.

## 2 FPGA et programmation

Les FPGA représentent une évolution majeure dans le domaine de la conception matérielle. Contrairement aux circuits intégrés spécifiques (ASIC), dont la fabrication nécessite des coûts de conception initiaux très importants, les FPGA permettent de configurer après fabrication une architecture matérielle adaptée à un besoin particulier. Cette flexibilité réduit considérablement les coûts et les délais de développement, notamment pour les systèmes nécessitant des évolutions fréquentes ou des productions en volumes limités.

La particularité principale des FPGA réside dans leur capacité de reconfiguration. Ils permettent d'exploiter un parallélisme spatial en instanciant directement plusieurs unités de calcul matérielles. Cette caractéristique en fait des accélérateurs matériels particulièrement adaptés à certaines applications industrielles nécessitant de fortes performances, comme le traitement du signal, l'intelligence artificielle, le calcul scientifique ou encore les systèmes embarqués temps réel.

Les FPGA sont composés de bloc logique reprogrammable permettant cette forte modularité, (voir Figure 1).

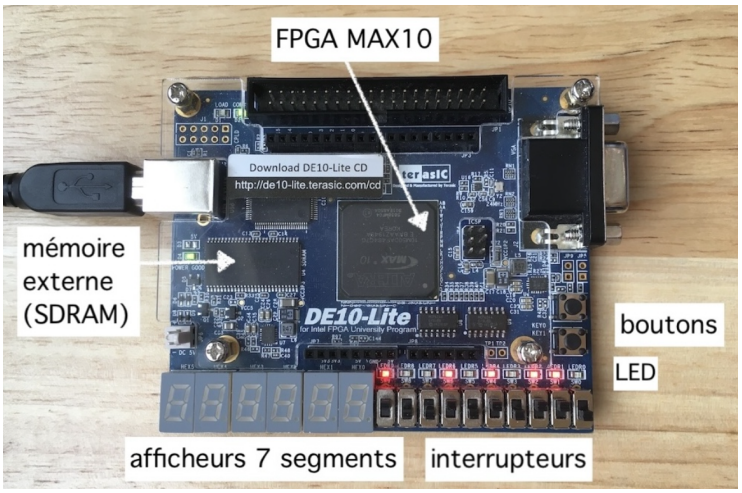



FIGURE 1 – Schéma d'un FPGA : Intel DE-10 lite

Dans le coeur d'une puce FPGA se trouvent les blocs logiques reprogrammables (voir Figure 2), qui constituent l'un des principaux intérêts de ces cartes. Dans l'architecture du FPGA utilisé durant ce stage, les blocs logiques sont constitués notamment de LUT permettant d'implémenter des fonctions logiques, ainsi que de registres permettant de mémoriser des états entre les cycles d'horloge. [2] Sans cela, le circuit serait purement combinatoire.

2. Look-Up Tables : une petite mémoire permettant de réaliser n'importe quelle fonction logique de quelques entrées.

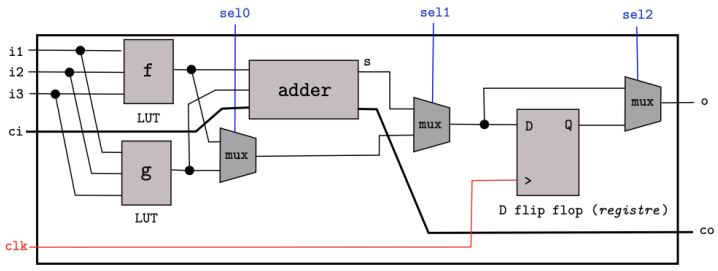



FIGURE 2 – Structure d'un élément logique avec additionneur intégré

Plusieurs notions sont essentielles pour analyser les performances et les benchmarks d'un processeur. Commençons par l'horloge et ses concepts clés. L'horloge agit comme le métronome du circuit : elle rythme le fonctionnement global et indique à quel moment le système a le droit d'actualiser son état. Chaque cycle d'horloge définit ainsi la fenêtre de temps allouée au circuit pour effectuer ses calculs. Au cours d'un même cycle, on distingue deux moments très précis, appelés les fronts d'horloge. Il y a le front montant et le front descendant. En règle générale, un circuit est conçu pour réagir à l'un ou l'autre de ces fronts (mais pas aux deux à la fois). C'est ce front déclencheur qui marque l'instant exact où les registres du circuit verrouillent les données et mettent à jour leur état.

Toutes puces éléctronique possède une fréquence. La carte DE10-Lite dispose notamment d'une source d'horloge de 50 MHz. Les registres cadencés par cette horloge peuvent capturer une nouvelle valeur à chaque front actif. Avec une horloge de 50 MHz, cela correspond à un front toutes les 20ns(1/fréquence), soit au maximum 50 millions de captures par seconde.

Cependant, l'exploitation de ces architectures nécessite traditionnellement l'utilisation de langages de description matérielle (HDL), tels que le VHDL ou le Verilog. Contrairement aux langages de programmation classiques, ces langages ne décrivent pas une succession d'instructions exécutées par un processeur, mais une architecture matérielle composée de composants logiques fonctionnant potentiellement en parallèle. Cette approche offre un contrôle très fin sur l'implémentation matérielle, mais introduit également une complexité importante et nécessite des connaissances spécifiques en conception numérique. L'objectif est donc de proposer des outils permettant de rapprocher la programmation FPGA des méthodes utilisées en programmation logicielle classique, tout en conservant un contrôle suffisant sur l'architecture générée..

Un des exemples les plus connus est HLS (High-level Synthesis)[3] afin de se concentrer sur l'aspect algorithmique et n'ont pas sur l'architecture du circuit. Nous pouvons notamment écrire dans un sous ensemble de C/C++ (pointeur et allocation dynamique parfois exclus) afin que l'outil de synthèse de haut niveau parse votre programme et le synthétise en HDL. Mais cette approche à une limite. Le contrôle de l'architecture par le concepteur du programme est

grandement limité et est délégué à l'outil de synthèse.

C'est dans cette perspective que s'inscrit Eclat. Ce langage propose un paradigme de programmation plus proche des langages logiciels traditionnels (OCaml-like) tout en conservant la possibilité de générer une architecture matérielle efficace. Le compilateur Eclat permet ainsi de traduire les programmes écrits dans ce langage en une description matérielle VHDL synthétisable, pouvant ensuite être implémentée sur un FPGA.

# 3 Expérimentations

## 3.1 Processeur LC-3

Pour répondre à notre question, nous allons reprendre un ancien projet de mon premier semestre de master. Ce projet portait sur la réalisation d'un processeur à but éducatif nommé LC-3. C'est un processeur relativement simple, comprenant environ 15 instructions de base. Ces dernières se décomposent en trois familles : les instructions arithmétiques et logiques, les instructions de chargement et rangement et les instructions de branchement (appelées aussi instructions de saut ou encore instructions de contrôle).

Les instructions arithmétiques et logiques du LC-3 sont au nombre de trois. Le LC-3 possède une instruction ADD pour l'addition, une instruction AND pour le et logique bit à bit et une instruction NOT pour la négation bit à bit. Le résultat de ces trois opérations est toujours placé dans un registre. Les deux opérandes peuvent être soit les contenus de deux registres soit le contenu d'un registre et une constante pour ADD et AND.

Les instructions de chargement et rangement comprennent des instructions (avec des noms commencent par LD) permettant de charger un registre avec une valeur et des instructions (avec des noms commencent par ST) permettant de ranger en mémoire le contenu d'un registre. Ces instructions se différencient par leurs modes d'adressage qui peut être immédiat, direct, indirect ou relatif. Les instructions de chargement sont LD, LDI, LDR, LEA et les instructions de rangement sont ST, STI et STR.

Les instructions de branchement comprennent les deux instructions de saut BR et JMP, les deux instructions d'appel de sous-routine JSR et JSRR, une instruction TRAP d'appel système (pas implementé), une instruction RET de retour de sous-routine et une instruction RTI de retour d'interruption (pas implementé).

De plus, le micro-processeur LC-3 dispose de 8 registres généraux 16 bits appelés R0...R7 et comme la plupart des micro-processeurs, le LC-3 dispose d'un compteur de programme PC et d'un registre d'instruction IR qui sont tous les deux des registres 16 bits.

Le LC-3 dispose de trois indicateurs de condition N, Z et P, regroupés dans un registre d'état de trois bits. Ils sont positionnés dès qu'une nouvelle valeur est mise dans un des registres généraux R0...R7. Ceci a lieu lors de l'exécution d'une instruction logique (NOT et AND), arithmétique (ADD) ou d'une instruction de chargement (LD, LDI, LDR et LEA). Ils indiquent respectivement si cette nouvelle valeur est négative (n), nulle (z) et positive (p). Ces indicateurs sont utilisés par l'instruction de branchement conditionnel BR pour savoir si le branchement doit être pris ou non.

La mémoire du LC-3 est organisée par mots de 16 bits. L'adressage du LC-3 est également de 16 bits. La mémoire du LC-3 est donc formée de $2^{16}$ mots de 16 bits, c'est-à-dire 128 KiB avec des adresses de `0x0000` à `0XFFFF` (voir Figure 5).

### 3.2 Implémentation du LC3 en Eclat

Pour la première implémentation du processeur en Eclat, j'ai choisi de privilégier une approche impérative, plus proche des langages de programmation classiques et plus pédagogique pour prendre en main Eclat. Les registres ainsi que la mémoire ont été représentés par des tableaux, qui constituent la principale structure de données mutable du langage. Les modifications sont effectuées en place, c'est-à-dire par réécriture directe des cases mémoire concernées, sans recopier l'ensemble de la structure. Le tableau représentant la mémoire contient des valeurs du type somme `value`, capable de représenter les différents éléments susceptibles d'être stockés en mémoire, tels qu'une instruction, une adresse, un caractère ou une constante. Cette première implémentation est présentée en annexe. Différents problèmes ont été rencontrés lors du développement du processeur. L'un des premiers problèmes rencontrés concernait l'extension de signe lors du redimensionnement d'un entier.
Par exemple, lors du passage d'un entier de 5 à 16 bits, le résultat obtenu n'était pas celui attendu. Nous avons notamment utilisé le programme suivant :

```
let main () =
    let x : int<5> = 20 in
    let y : int<16> = int_resize<<16>>(x) in y
;;
```

Ce programme renvoyait systématiquement la valeur : 20 20 20 20 ...

Le problème provenait de la fonction de redimensionnement présente dans le code VHDL. Celle-ci utilisait systématiquement une conversion en `unsigned`. Lors d'une extension vers une taille supérieure, cette conversion ne permettait donc pas de conserver correctement le signe de la valeur.

Pour corriger ce problème, la fonction de redimensionnement a été modifiée afin de distinguer le cas d'une réduction de taille de celui d'une extension. Lorsqu'un entier est agrandi, la conversion utilise désormais le type `signed`, ce qui permet de réaliser une extension de signe. Lorsqu'un entier est réduit, les bits de poids faible sont conservés afin d'effectuer la troncature.

Cette modification permet ainsi d'obtenir le comportement attendu lors du redimensionnement d'entiers signés.

### 3.3 Résultats des benchmarks

J'ai commencé par réaliser un premier circuit simple permettant d'allumer des groupes de LED en fonction de l'état des boutons-poussoirs. Chaque bouton permettait ainsi d'allumer un groupe de cinq LED. Cette première expérimentation avait pour objectif de me familiariser avec la chaîne de compilation d'un langage de description matérielle, depuis le code Eclat jusqu'à la génération du fichier `.sof`. Ce fichier décrit le circuit synthétisé et permet ensuite de le programmer sur la carte FPGA.

Le circuit synthétisé à partir de cette première expérimentation utilisait un seul élément logique, sur les 49 760 éléments disponibles sur la carte.

J'ai ensuite réalisé une implémentation fonctionnelle du processeur LC-3, en utilisant des vecteurs pour représenter les différentes structures de données. Du point de vue de l'écriture du programme, cette approche était relativement similaire à l'implémentation impérative. Cependant, la gestion des structures de données posait un problème différent. Alors que la version impérative modifiait les tableaux en place, l'approche fonctionnelle reposait sur leur immuabilité. Ainsi, chaque modification d'un vecteur d'instructions ou d'un autre vecteur nécessitait la création d'une nouvelle copie de celui-ci. Le coût ne se situait donc plus principalement au niveau de la recopie de la mémoire, mais au niveau de l'espace mémoire nécessaire pour conserver ces différentes versions.

Une fois cette version développée, les résultats obtenus étaient cependant nettement meilleurs que ceux de la version impérative. Pour un même problème algorithmique, correspondant à la résolution des tours de Hanoï avec trois disques, le nombre de cycles est passé d'environ 5 225 à 500, soit un gain d'un facteur proche de 10.

Je me suis ensuite intéressé à une approche hybride, cherchant à combiner les avantages des approches impérative et fonctionnelle. Dans cette version, les instructions étaient stockées dans des tableaux, tandis que les petites structures de données étaient conservées sous forme de vecteurs. La recopie de petits vecteurs reste en effet limitée et ne représente pas un coût aussi important que la recopie de grandes structures. Cette approche a permis d'obtenir environ 1 000 cycles pour le même problème, soit un gain d'un facteur proche de 5 par rapport à la version impérative, tout en restant environ deux fois moins performante que la version fonctionnelle pure.

Ces résultats montrent que chaque approche possède ses propres avantages et inconvénients. Il n'existe donc pas de structure de données ou de style de programmation universellement optimal. Le choix dépend notamment de la taille des données manipulées et des contraintes d'exécution. Une approche fonctionnelle peut être intéressante pour de petits programmes ou de petites structures de données, notamment grâce à l'absence de modifications concurrentes et à la simplicité de raisonnement qu'elle offre. À l'inverse, une approche impérative utilisant des tableaux peut être plus adaptée à des programmes plus importants, pour lesquels la recopie de grandes structures de données entraînerait un coût mémoire trop élevé.

J'ai également réalisé un benchmark portant sur le nombre d'éléments lo-

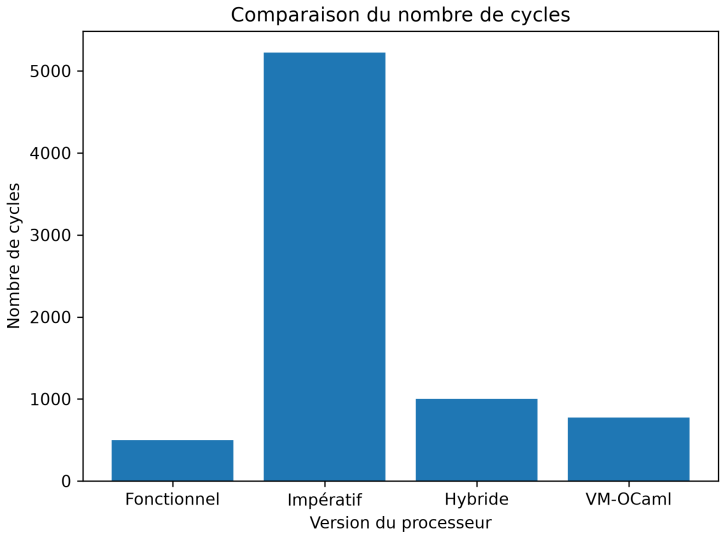



FIGURE 3 – Comparaison des performances des implémentations en fonction du nombre de cycles

giques utilisés par la version fonctionnelle. Cette mesure s'est cependant révélée difficile à effectuer. Dans un premier temps, la synthèse Quartus prenait un temps très important et ne produisait pas de résultat exploitable. J'ai alors essayé de réduire la taille des vecteurs utilisés dans le circuit, en passant notamment d'une taille de 200 à une taille de 10 éléments.

Malgré cette réduction, la synthèse a pris près de dix minutes et le circuit obtenu nécessitait plus de 18 846 éléments logiques. Il n'a donc pas été possible d'obtenir un résultat représentatif pour la version fonctionnelle avec des vecteurs de taille 200. Un tel circuit ne pouvait en pratique pas être synthétisé dans des conditions raisonnables avec Quartus. J'ai donc décidé de ne pas retenir le nombre d'éléments logiques comme critère de comparaison pour cette version, faute de résultats exploitables.

Enfin, j'ai comparé les performances de ces différentes implémentations avec celles de programmes équivalents exécutés dans des langages généralistes, notamment OCaml et la machine virtuelle OCaml implementé en Eclat, afin d'obtenir un ordre de grandeur des différences de vitesse. Pour ces langages, la comparaison ne peut plus être effectuée directement en nombre de cycles matériels. J'ai donc utilisé le temps d'exécution en espace utilisateur, mesuré avec le simulateur Eclat. Cette mesure permet de comparer les temps d'exécution des différentes versions, même si les modèles d'exécution ne sont pas identiques à ceux d'un circuit matériel.

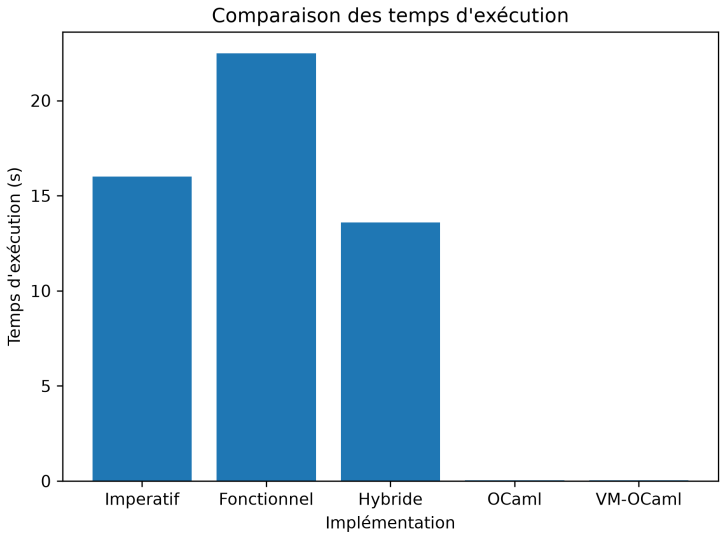



FIGURE 4 – Comparaison des temps d'exécution des implémentations en Eclat, OCaml et sur la VM-OCaml

# 4 Conclusion

Les expérimentations réalisées au cours de ce stage ont permis d'étudier l'utilisation d'Eclat pour la programmation de circuits FPGA, notamment à travers l'implémentation d'un processeur LC-3 selon plusieurs styles de programmation.

La comparaison des approches fonctionnelle, impérative et hybride montre que le choix du style de programmation peut avoir une influence importante sur le comportement du circuit généré. Dans le cas étudié, l'implémentation fonctionnelle nécessite environ 500 cycles pour résoudre le problème des tours de Hanoï, contre environ 5,225 pour l'implémentation impérative et environ 1,000 pour l'implémentation hybride. L'approche fonctionnelle nécessite donc nettement moins de cycles pour ce benchmark, tandis que l'approche hybride se situe entre les deux.

Ces résultats mettent cependant en évidence un compromis entre les performances temporelles et les ressources matérielles nécessaires à la réalisation du circuit. L'approche fonctionnelle permet ici de réduire le nombre de cycles nécessaires à l'exécution, mais la représentation des données par des structures immuables peut entraîner la création de nombreuses versions d'une même structure. Cette représentation peut alors se traduire par une utilisation plus importante des ressources logiques du FPGA. À l'inverse, l'approche impérative permet de modifier les structures de données en place, ce qui peut limiter certaines duplications matérielles, au prix d'un nombre de cycles plus important dans le cas étudié.

La représentation des structures de données constitue donc un élément im-

portant dans le comportement du circuit généré. Les différences observées entre les tableaux utilisés dans l'approche impérative et les vecteurs utilisés dans l'approche fonctionnelle montrent que le choix de la structure de données peut également avoir un impact sur les ressources nécessaires à la réalisation du circuit.

L'approche hybride apparaît quant à elle comme une solution intermédiaire intéressante. Dans notre cas, elle permet d'obtenir environ 1,000 cycles, soit un résultat meilleur que l'approche impérative tout en limitant certaines des contraintes liées à l'utilisation systématique de structures fonctionnelles.

Ces résultats montrent ainsi qu'il n'existe pas nécessairement un style de programmation universellement meilleur. Le choix entre une approche fonctionnelle, impérative ou hybride doit dépendre de l'application considérée, de la taille et de la nature des données manipulées ainsi que des contraintes matérielles.

Enfin, les résultats obtenus restent limités au processeur LC-3 et aux benchmarks réalisés au cours du stage. Une étude plus complète de la fréquence maximale atteignable et de l'utilisation des différentes ressources du FPGA permettrait de mieux caractériser les compromis entre les différentes approches. Des expérimentations sur d'autres applications permettraient également de déterminer si les tendances observées se généralisent à d'autres circuits.

## Références


[1] Olivier Sentieys and Arnaud Tisserand. Architectures reconfigurables FPGA. In Techniques de l'Ingénieur, editor, *Technologies logicielles Architectures des systèmes*, volume H 1 196, pages 1–22. Techniques de l'Ingénieur, August 2012.

[2] Loïc Sylvestre. Programmation sûre, expressive et efficace de circuits fpga. 2025.

[3] Daouda Diakité. Synthèse de haut niveau sur fpga pour problèmes inverses : application à la tomographie et à la radioastronomie. 2023.

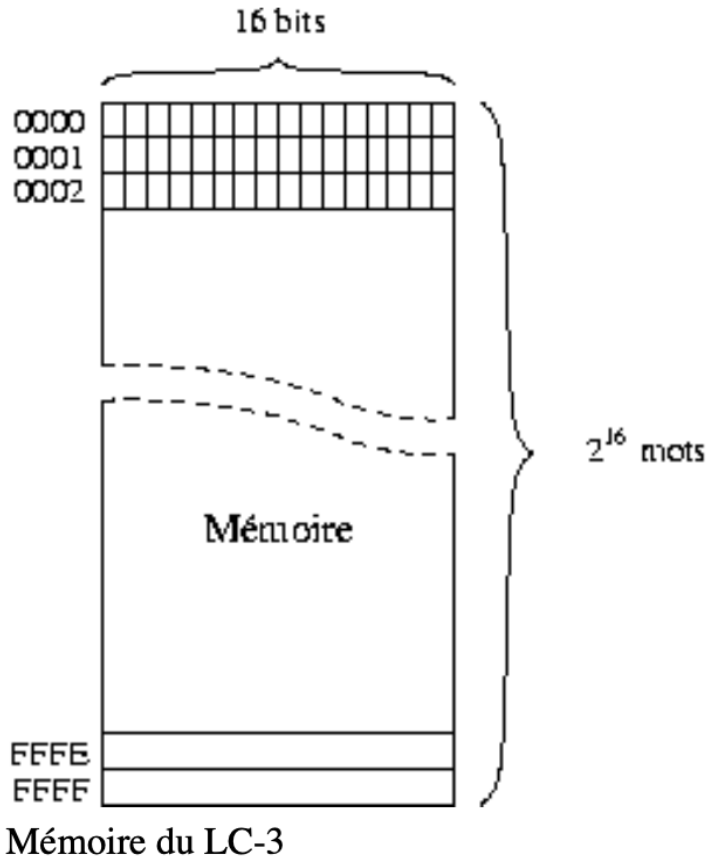



FIGURE 5 – Représentation de la mémoire LC-3

| Syntaxe | Action | nzp | Codage | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Op-code | | | | Arguments | | | | | | | | | | | |
| | | | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| NOT DR,SR | DR ← not SR | * | 1 | 0 | 0 | 1 | DR | | | SR | | | 1 | 1 | 1 | 1 | 1 | 1 |
| ADD DR,SR1,SR2 | DR ← SR1 + SR2 | * | 0 | 0 | 0 | 1 | DR | | | SR1 | | | 0 | 0 | 0 | SR2 | | |
| ADD DR,SR1,Imm5 | DR ← SR1 + SEXT(Imm5) | * | 0 | 0 | 0 | 1 | DR | | | SR1 | | | 1 | Imm5 | | | | |
| AND DR,SR1,SR2 | DR ← SR1 and SR2 | * | 0 | 1 | 0 | 1 | DR | | | SR1 | | | 0 | 0 | 0 | SR2 | | |
| AND DR,SR1,Imm5 | DR ← SR1 and SEXT(Imm5) | * | 0 | 1 | 0 | 1 | DR | | | SR1 | | | 1 | Imm5 | | | | |
| LEA DR,label | DR ← PC + SEXT(PCoffset9) | * | 1 | 1 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| LD DR,label | DR ← mem[PC + SEXT(PCoffset9)] | * | 0 | 0 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| ST SR,label | mem[PC + SEXT(PCoffset9)] ← SR | | 0 | 0 | 1 | 1 | SR | | | PCoffset9 | | | | | | | | |
| LDR DR,BaseR,Offset6 | DR ← mem[BaseR + SEXT(Offset6)] | * | 0 | 1 | 1 | 0 | DR | | | BaseR | | | Offset6 | | | | | |
| STR SR,BaseR,Offset6 | mem[BaseR + SEXT(Offset6)] ← SR | | 0 | 1 | 1 | 1 | SR | | | BaseR | | | Offset6 | | | | | |
| LDI DR,label | DR ← mem[mem[PC + SEXT(PCoffset9)]] | * | 1 | 0 | 1 | 0 | DR | | | PCoffset9 | | | | | | | | |
| STI SR,label | mem[mem[PC + SEXT(PCoffset9)]] ← SR | | 1 | 0 | 1 | 1 | SR | | | PCoffset9 | | | | | | | | |
| BR[n][z][p] label | Si (cond) PC ← PC + SEXT(PCoffset9) | | 0 | 0 | 0 | 0 | n | | z | p | PCoffset9 | | | | | | | |
| NOP | No Operation | | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 0 0 0 0 0 0 0 0 | | | | | | | | |
| JMP BaseR | PC ← BaseR | | 1 | 1 | 0 | 0 | 0 0 0 | | | BaseR | | | 0 0 0 0 0 0 | | | | | |
| RET (≡ JMP R7) | PC ← R7 | | 1 | 1 | 0 | 0 | 0 0 0 | | | 1 1 1 | | | 0 0 0 0 0 0 | | | | | |
| JSR label | R7 ← PC; PC ← PC + SEXT(PCoffset11) | | 0 | 1 | 0 | 0 | 1 | PCoffset11 | | | | | | | | | | |
| JSRR BaseR | R7 ← PC; PC ← BaseR | | 0 | 1 | 0 | 0 | 0 | 0 | 0 | BaseR | | | 0 0 0 0 0 0 | | | | | |
| RTI | cf. interruptions | | 1 | 0 | 0 | 0 | 0 0 0 0 0 0 0 0 0 0 0 0 | | | | | | | | | | | |
| TRAP Trapvect8 | R7 ← PC; PC ← mem[Trapvect8] | | 1 | 1 | 1 | 1 | 0 0 0 0 | | | | | | Trapvect8 | | | | | |
| Réservé | | | 1 | 1 | 0 | 1 | | | | | | | | | | | | |

FIGURE 6 – Ensemble du jeu d'instruction du LC-3 (ps : les * indiquent si l'instruction mets à jour les flags NZP)